

Executive Summary of Project 4

-Ananya Asthana

Assignment Overview

The purpose of this assignment was to add fault tolerance and achieve consensus amongst our server replicas by using the Paxos protocol. It aimed to help us understand the roles of Acceptor, proposer and learner in this protocol. Another key part was simulating failures of all the 3 roles to check what happens when consensus is not reached and to demonstrate the fault tolerant nature of Paxos. This involved introducing probabilistic failures in both the prepare and accept phases for Acceptors, as well as learning delays or losses in the Learner and operation disruption in the Proposer. We needed to ensure that each server had its own local instances of the paxos role. Additionally, we had to maintain features from prior assignments such as multithreaded clients, data prepopulation, and replica management, ensuring they continued to function correctly with the Paxos logic integrated. Overall, the assignment aimed to simulate a realistic fault-tolerant distributed key-value store and highlight the internal working of consensus under unreliable conditions.

Technical Impression

My experience was definitely productive, as it helped me understand what paxos is. I got an in-depth look into the roles needed and the relationships between them, as well as the order or flow of control from the time a client sends a request to the time consensus it achieved and operation is carried out, or it fails.

The main difficulty I faced was with simulating failures and ensuring they are occurring at a desirable rate wherein my process is not disrupted too much but it is also challenged by failures to test it. I also took some time to understand the design, whether I wanted the proposer to call all the acceptors and learners or just its own. In the end, I chose for each Proposer to talk to all Acceptors and Learners across replicas rather than limiting interactions locally. This allowed me to better model distributed consensus and understand quorum behavior. Debugging was sometimes tricky because of concurrent logs, but adding structured log messages with clear thread names helped me trace the Paxos flow more clearly. I also had to ensure that learner failures didn't impact the final result, and I saw firsthand how Paxos can still complete successfully even if some learners drop out. Debugging log interleaving and consensus breakdowns under failure scenarios gave me a better grasp of real-world distributed system resilience.